SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA1017	**Course Title:**	Software Engineering
CO Assessed:	CO4	**Assessment:**	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	**Total Marks:**	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	LOSHINI.L	**Reg. No.:**	192571015
Date:	18/08/26	**Duration:**	60 minutes

Code of Conduct

I LOSHINI.L 192571015 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: LOSHINI.L_____

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25%)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15%)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	Mark
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4

Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25

Problem Statement

Improper segregation of municipal waste reduces recycling efficiency and increases landfill usage. Existing waste processing facilities often rely on manual sorting, leading to inconsistent results and higher operational costs. An AI-powered waste segregation system should identify different categories of waste using computer vision, automate sorting through robotic mechanisms, monitor recycling statistics, and generate waste management reports. The platform should improve recycling efficiency while reducing environmental pollution.

1. Requirement Analysis and CI/CD Needs

The AI-Based Smart Waste Segregation and Recycling Automation System is a hybrid software-hardware platform combining a computer vision inference engine, robotic sorting control firmware, a backend analytics service, and a web/mobile reporting dashboard. This mix of components introduces CI/CD requirements that go beyond a typical web application.

1.1 Key System Components

- **Computer Vision Module:** A trained deep learning model (e.g., CNN-based classifier) that identifies waste categories — plastic, paper, metal, glass, organic, e-waste — from camera feeds on the conveyor belt.
- **Robotic Control Module:** Embedded/edge software (running on microcontrollers or edge devices such as Jetson Nano/Raspberry Pi) that receives classification output and drives robotic arms/actuators to sort waste into designated bins.
- **Backend & Analytics Service:** APIs and databases that log every sorting event, compute recycling statistics, and store historical data for reporting.
- **Reporting Dashboard:** A web/mobile application used by facility managers and municipal authorities to view recycling efficiency, contamination rates, and generate waste management reports.
- **IoT/Edge Integration Layer:** Middleware (e.g., MQTT) connecting sensors, cameras, and robotic actuators to the cloud backend.

1.2 CI/CD Needs Identified

- **Frequent, safe model updates:** The vision model must be retrained periodically as new waste types/packaging appear, so the pipeline needs a path to validate and deploy new model versions without breaking production sorting.
- **Multi-artifact builds:** The pipeline must build and version three distinct artifact types — the AI model, the edge/robotics firmware, and the backend/dashboard application — each with its own test and deployment path.
- **Hardware-in-the-loop testing:** Robotic control logic must be tested in a simulated environment before being pushed to physical edge devices, since faulty deployments can damage hardware or misroute waste.
- **High reliability and safety:** Since robotic arms and conveyor systems are physically operating machinery, the pipeline must include strict quality gates, staged rollout, and fast rollback to avoid unsafe or incorrect sorting actions.
- **Data and model quality checks:** Because model accuracy directly impacts recycling efficiency, the pipeline must validate model performance (accuracy, precision/recall per waste class) before promotion to production.

- **Scalability across facilities:** The platform will be deployed at multiple municipal waste facilities, so deployment must be repeatable, environment-agnostic (via containers), and centrally monitored.
- **Regulatory and environmental reporting integrity:** Waste management reports feed into compliance and environmental audits, so the reporting service requires rigorous testing and traceable deployments.

1.3 System Component Mindmap

The mindmap below summarizes how the major system components relate to one another.

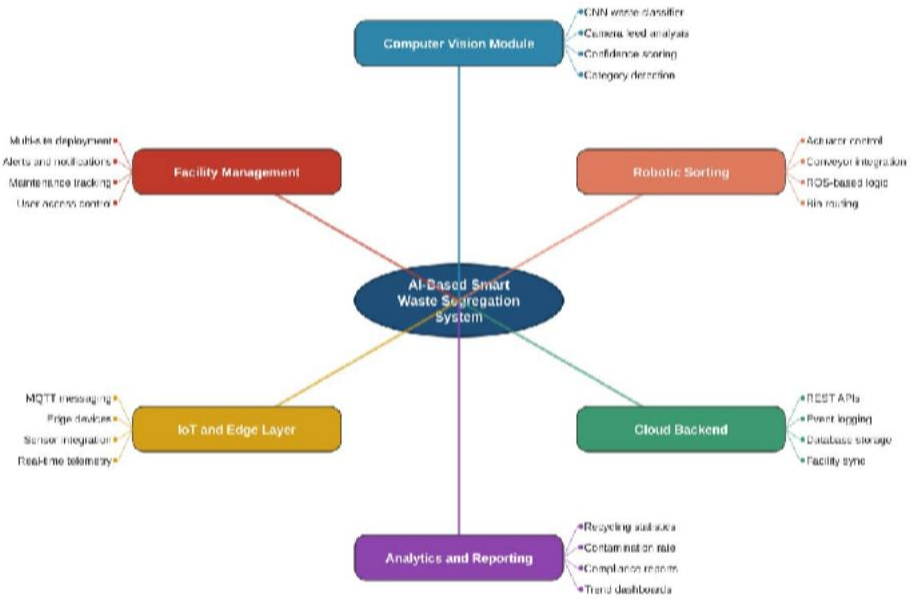

Figure A: System Component Mindmap — AI-Based Smart Waste Segregation System

2. CI/CD Pipeline Architecture and Workflow

The pipeline follows a **step-by-step process** starting from source code management and moving through building, automated testing, security and quality checks, AI model validation, packaging, and staged deployment. After deployment, the system is continuously monitored to ensure proper performance. If any major error or performance issue is detected, an **automatic rollback** restores the previous stable version. This workflow ensures that updates to the smart waste segregation system are **safe, reliable, secure, and efficient**.

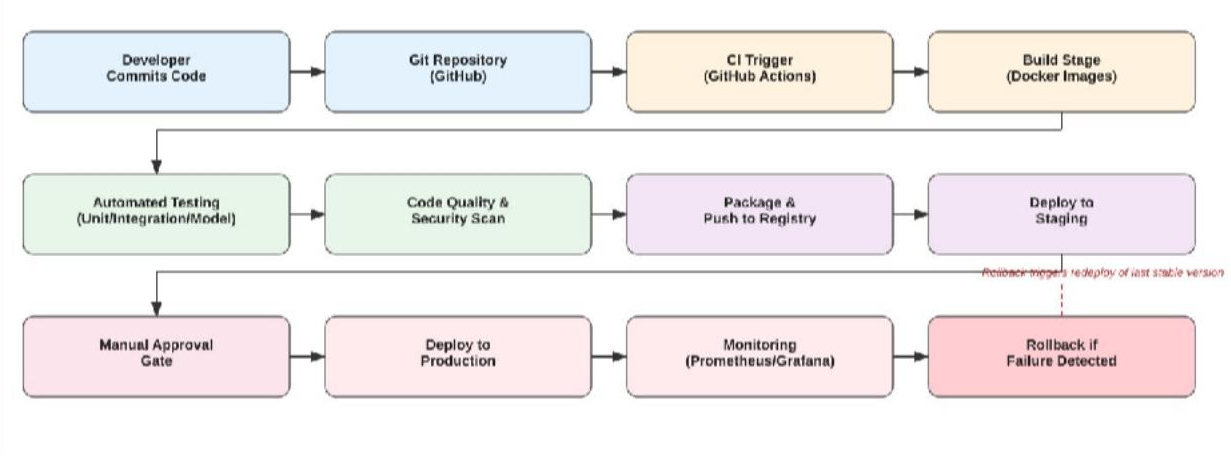

Figure 1: CI/CD Pipeline Workflow — AI-Based Smart Waste Segregation System

2.1 Pipeline Stages Overview

Stage	Purpose
1. Source Code Management	Developers commit code (model training scripts, robotic control firmware, backend/dashboard code) to separate branches in a Git monorepo or linked repositories.
2. CI Trigger	A push or pull request automatically triggers the CI pipeline via webhooks.
3. Build	Backend/dashboard code is compiled/bundled; the AI model is retrained or loaded; firmware is cross-compiled for the target edge hardware; Docker images are built for each service.
4. Automated Testing	Unit tests, integration tests, model validation tests, and simulated robotic control tests run automatically.
5. Code Quality & Security Analysis	Static analysis, linting, dependency vulnerability scanning, and model bias/data-drift checks are performed.
6. Packaging	Validated artifacts (Docker images for backend/dashboard, versioned model files, signed firmware binaries) are packaged and pushed to a registry.
7. Deployment (Staging → Production)	Artifacts are deployed first to a staging environment that mirrors production (including a simulated conveyor/robot rig), then promoted to production after approval.
8. Monitoring & Rollback	Live sorting accuracy, robotic actuation success rate, and system health are monitored continuously; failures trigger automatic rollback to the last stable version.

3. Tools and Technologies Selected

Each stage uses tools chosen for reliability, strong community support, and suitability for a mixed AI/robotics/web system.

Pipeline Stage	Tool(s) Selected	Justification
Source Code Management	Git + GitHub	Industry-standard distributed version control with strong branching, pull-request review, and webhook support for triggering CI.
CI/CD Orchestration	GitHub Actions (alt: Jenkins)	Native integration with GitHub, YAML-based pipeline-as-code, large marketplace of pre-built actions, and free tier suitable for academic/pilot deployment.
Build & Containerization	Docker, Docker Compose	Ensures the backend, dashboard, and inference service run consistently across development, staging, and production, and across multiple facility deployments.
AI Model Training/Versioning	TensorFlow/PyTorch, DVC (Data Version Control), MLflow	TensorFlow/PyTorch for training the waste-classification CNN; DVC and MLflow version datasets, model artifacts, and track experiment metrics so model changes are reproducible and auditable.
Edge/Robotics Build	PlatformIO / cross-compilation toolchain (for microcontrollers), ROS (Robot Operating System) for actuator control	PlatformIO standardizes firmware builds for edge boards; ROS provides a mature, testable framework for robotic arm/actuator control logic.
Unit & Integration Testing	PyTest (backend/model), Jest (dashboard), ROS test framework / Gazebo simulator (robotics)	PyTest and Jest are standard, well-supported frameworks for Python and JavaScript; Gazebo allows robotic sorting logic to be tested in simulation before touching physical hardware.
Code Quality & Static Analysis	SonarQube, ESLint, Pylint	Detects code smells, complexity issues, and style violations early, keeping the multi-language codebase maintainable.
Security Scanning	OWASP Dependency-Check, Trivy (container image scanning), Bandit (Python security linting)	Identifies vulnerable dependencies and insecure container images before deployment, which is critical since the system controls physical machinery.
Artifact Registry	Docker Hub / GitHub Container Registry, MLflow Model Registry	Central, versioned storage for container images and trained models, enabling traceable rollback to any prior version.
Deployment	Kubernetes (production orchestration), Ansible (edge device configuration)	Kubernetes manages scalable, self-healing deployment of backend/dashboard services across facilities; Ansible automates configuration of edge/robotic controllers.

4. Automated Testing Strategy

Because the system spans AI inference, robotics, and web services, testing is layered to catch issues at the appropriate stage before they reach physical hardware or end users.

4.1 Testing Layers

Testing Layer	What It Covers
Unit Testing	Individual functions tested in isolation — image preprocessing, confidence-thresholding logic, API handlers, and report calculations. Runs on every commit.
Model Validation Testing	The vision model is evaluated against a held-out labeled dataset per waste category (≥95% precision/recall per class required). Confusion-matrix regression checks prevent accuracy drops versus the current production model.
Integration Testing	Verifies the vision module correctly hands off results to the robotic control module and that sorting events are logged and reflected in recycling statistics.
Simulation / Hardware-in-the-Loop	Robotic arm and conveyor logic is tested in a Gazebo/ROS simulation before touching physical hardware, catching unsafe actuator commands early.
End-to-End (E2E) Testing	Full pipeline test from simulated camera feed to correct bin actuation to updated dashboard statistics, run in staging.
Performance & Load Testing	Validates real-time inference latency against conveyor speed, and backend/dashboard capacity under concurrent multi-facility load.
Regression Testing	A fixed suite, including past misclassified edge cases, runs on every model or code change to prevent reintroducing old defects.

4.2 Testing Integration in the Pipeline

All test suites run automatically within the CI stage immediately after build. A merge/deployment is blocked if any test suite fails, if model accuracy falls below the defined threshold, or if the security scan reports high/critical vulnerabilities. Test results and coverage reports are published as pipeline artifacts and linked in pull-request checks.

5. Deployment Strategy

Deployment follows a three-environment promotion model, with additional care taken for the physical robotic components.

Environment	Purpose	Deployment Details
Development	Used by developers for day-to-day feature work and quick iteration.	Auto-deployed on every commit to feature branches. Uses mock camera feeds and a simulated robot (Gazebo) — no physical actuation.
Testing / Staging	Mirrors production configuration, including a physical or high-fidelity simulated sorting rig, for final validation.	Auto-deployed on merge to the main/develop branch after all tests pass. Full end-to-end tests, load tests, and a manual QA sign-off are performed here before promotion.
Production	Live municipal waste facility deployments controlling real robotic sorting hardware.	Deployed only after manual approval by a release manager. Uses a blue-

Environment	Purpose	Deployment Details
		green or canary deployment strategy: the new version runs alongside the current stable version on a subset of sorting lines/facilities first, and is gradually rolled out once monitoring confirms stable accuracy and actuation success rates.

5.1 Deployment Approach for AI Models vs. Application Code

- **Application/backend/dashboard code:** Deployed via rolling updates in Kubernetes, ensuring zero downtime for the reporting dashboard and APIs.
- **AI model artifacts:** Deployed via canary release — the new model handles a small percentage of live classification traffic while its accuracy is compared against the current production model before full rollout.
- **Robotic firmware:** Deployed to edge devices in controlled batches (facility-by-facility) using Ansible, with each batch monitored for actuation errors before proceeding to the next.

6a. CI/CD Pipeline Focus Areas — Mindmap

The mindmap below summarizes the key focus areas covered by the pipeline: testing, security, deployment, monitoring, quality gates, and notifications.

Figure B: CI/CD Pipeline Key Focus Areas Mindmap

6. Security, Quality Checks, Notifications, and Rollback Mechanisms

6.1 Security Practices

Practice	Detail
Secrets Management	API keys, database credentials, and device certificates are stored in a secrets manager (GitHub Actions Secrets/HashiCorp Vault) and never committed to source control.
Container Image Scanning	Images are scanned with Trivy for known vulnerabilities before registry push; builds fail on critical/high severity findings.
Dependency Scanning	OWASP Dependency-Check runs on every build to catch vulnerable libraries in the backend and dashboard.
Encrypted Device Communication	Edge devices (cameras/robotic controllers) communicate with the cloud over TLS/MQTT-over-TLS and authenticate using signed certificates.
Access Control	Role-based access control (RBAC) restricts who can approve production deployments or modify robotic control parameters.

6.2 Quality Gates

Gate	Requirement
Code Coverage	Minimum threshold (e.g., 80%) enforced before merge.
Static Analysis	SonarQube quality gate must pass — no new critical bugs or code smells — before packaging.
Model Accuracy	New model must meet or exceed the accuracy/precision/recall of the currently deployed model on the validation dataset.
Manual QA Sign-off	Required in staging before any production promotion.

6.3 Notifications

Channel	Trigger
Slack / Teams	Automated alerts for build failures, failed test suites, security scan findings, and deployment outcomes.
Email	Sent to the release manager whenever a production deployment requires manual approval.
Grafana Alerting	Real-time notification to the operations team if sorting accuracy drops below threshold or actuation errors spike.

6.4 Rollback Strategy

Scenario	Rollback Mechanism
General Artifact Rollback	Every deployed artifact (backend image, model version, firmware binary) is immutably versioned, allowing instant redeployment of the last known-good version.
Backend/Dashboard Failure	Kubernetes rolling deployments automatically roll back if health checks fail after a new release.
AI Model Accuracy Drop	If live classification accuracy (via sampled human-verified outcomes) drops below threshold during canary rollout, the system automatically reverts to the previous model version.
Robotic Firmware Failure	Each facility batch deployment has a health-check window; failing actuation success criteria halts rollout and reverts the batch to the previous firmware.
Post-Incident Follow-up	All rollbacks are logged and automatically trigger a post-incident report for root-cause analysis.

7. Pipeline Diagram and Workflow Explanation

The diagram (Figure 1) shows the complete journey of a change from code commit to production deployment, organized into three logical rows.

7.1 Row 1 – Integration and Build

A developer commits code (model training script, robotic control logic, or backend/dashboard feature) to the Git repository hosted on GitHub. This push automatically triggers the CI pipeline via GitHub Actions, which then builds the relevant artifact — compiling the backend/dashboard, retraining or loading the AI model, or cross-compiling robotic firmware — and packages each into Docker images where applicable.

7.2 Row 2 – Verification and Packaging

The build artifacts flow into automated testing, covering unit, integration, model-validation, and simulated robotic control tests. Passing artifacts then undergo code quality and security scanning (SonarQube, Trivy, dependency checks). Only artifacts that clear both gates are packaged and pushed to the artifact/model registry, then deployed to the staging environment for full end-to-end validation.

7.3 Row 3 – Controlled Release and Recovery

After staging validation succeeds, a manual approval gate ensures a release manager reviews and authorizes the change before it reaches physical waste-sorting hardware. Approved releases are deployed to production using canary/blue-green strategies. Prometheus and Grafana continuously monitor sorting accuracy, actuation success rate, and system health. If monitoring detects a failure or degraded performance, the rollback mechanism (dashed red arrow) automatically redeploys the last stable version to staging/production, minimizing downtime and preventing incorrect physical sorting actions.

This closed-loop design — build, test, deploy, monitor, and roll back — ensures the AI-Based Smart Waste Segregation and Recycling Automation System can be updated frequently and safely, while protecting the physical robotic hardware and maintaining consistent, high-accuracy recycling performance.

8. Conclusion

- The proposed **Continuous Integration and Continuous Deployment (CI/CD) pipeline** is specifically designed to handle the combined software, artificial intelligence, and hardware requirements of the **AI-Based Smart Waste Segregation and Recycling Automation System**. Unlike a traditional software application, this system depends on AI models, cameras or sensors, robotic mechanisms, and automated waste-sorting equipment. Therefore, every software or AI update must be carefully tested before it is introduced into real-world municipal waste-processing facilities.
- The pipeline begins with **continuous integration**, where developers regularly upload changes to the source-code repository. Whenever new code, AI-model updates, or configuration changes are submitted, automated processes are triggered. The system performs **code quality checks, dependency verification, unit testing, and integration testing** to identify errors at an early stage. This prevents faulty code from reaching the production environment.
- A major feature of the pipeline is **AI model validation**. Since the system uses AI to identify and classify different types of waste such as plastic, paper, metal, glass, and organic materials, every new model version must be evaluated before deployment. The pipeline can automatically check important performance measures such as **accuracy, precision, recall, F1-score, and classification errors**. The new model is deployed only when it satisfies predefined quality requirements. This helps prevent an inaccurate model from causing incorrect waste segregation.
- The pipeline also includes **hardware-in-the-loop testing**, which is particularly important for this project. The AI system may control physical components such as conveyor belts, robotic arms, sensors, cameras, and sorting mechanisms. Before deploying an update to actual equipment, the software can be tested with simulated or connected hardware. This verifies that commands are generated correctly and safely. For example, if the AI identifies a plastic bottle, the system should send the correct control signal to the appropriate sorting mechanism without causing unsafe movement or equipment failure.
- To reduce deployment risks, the system follows a **staged deployment strategy**. New versions can first be tested in a development environment, followed by a testing environment and then a limited pilot or municipal facility. After confirming that the updated software and AI model work correctly, the release can gradually be extended to additional facilities. This approach prevents a single faulty update from affecting the entire waste-management network.
- The pipeline also implements **security and quality gates**. Security scanning can identify vulnerable dependencies, unauthorized code changes, and other potential threats before deployment. Quality gates ensure that software, AI models, and configuration files meet predefined standards. If any critical test fails, the deployment process is automatically stopped until the problem is corrected.
- After deployment, **continuous monitoring** is used to observe system performance in real time. The system can monitor AI classification accuracy, waste-sorting efficiency, sensor status, processing speed, equipment errors, and system availability. AI performance can also be monitored for **model drift**, where the model becomes less accurate because the type or appearance of waste changes over time. Monitoring allows administrators to identify problems quickly and take corrective action.

- Another important feature is **automatic rollback**. If a newly deployed software version or AI model causes unexpected errors, reduced sorting accuracy, or hardware problems, the pipeline can automatically return the system to the previous stable version. This minimizes downtime and prevents serious operational failures in municipal facilities.
- Overall, the CI/CD pipeline provides a **reliable, secure, and scalable method for continuously improving the smart waste-management system**. By combining conventional software testing with AI model validation, hardware testing, staged deployment, security checks, monitoring, and automatic rollback, the pipeline ensures that updates are introduced safely. This supports the main objectives of the project by improving **waste segregation accuracy, recycling efficiency, system reliability, and environmental sustainability** across multiple municipal facilities.